INSPECTAI / NL2SQL BACKEND ENGINEERING REPORT

Regex-Driven Pipelines → Explicit Tools

A session-long audit of every regex-gated intent-resolution mechanism in the agent orchestration loop — what became a tool, what didn't, and why.

Scope: `khidmah-nl2sql-prototype/backend/src`

Session date: 2026-08-05

Phases covered: 0 through 6 (framework build-out, two new LLM-callable tools, four honest evaluations, five real bugs fixed)

Verification: unit tests plus live runs against the real database and real Ollama models (qwen3.5:9b / qwen2.5-coder) at every phase boundary

1 Executive Summary

The NL2SQL agent had a recurring architecture pattern: a piece of code would *guess* what the user meant by regex-matching the raw question text, then inject unenforced prose into the reasoning LLM's system prompt hoping it would comply. A live bug — two facilities sharing the same `audit_plan` title, silently resolved to the wrong one — exposed this concretely. Auditing the codebase surfaced a family of similarly-shaped regex pipelines. Today's session worked through that entire family, one mechanism at a time, converting what could honestly be converted into explicit, LLM-callable tools, and fixing what regex-parsing bugs surfaced along the way in the mechanisms that were *not* converted.

2	**4**	**8**	**400**
NEW LLM TOOLS BUILT	MECHANISMS EVALUATED, KEPT AS-IS (WITH FIXES)	REAL BUGS FOUND & FIXED	TESTS PASSING (FROM 0 FOR THIS EFFORT)

> **The honest headline on "regex reduction":** only two of the six mechanisms actually had their regex parsing *eliminated* — replaced by the LLM supplying structured data it alone can extract from free text. The other four were deliberately *not* converted, after direct evidence-based evaluation, because there was nothing in them for an LLM tool call to usefully supply — the code already has everything it needs from session state or the literal question text. Converting those anyway would have added a mandatory extra LLM round trip to the common case for no correctness gain. Section 3 gives the full accounting.

2 What Changed, at a Glance

Phase	Mechanism	Outcome	Headline result
0 + 1	Ordinal follow-ups ("the first one")	TOOL BUILT	New `resolve_ordinal_reference` tool + shared tool-declaration framework. 2 real bugs found live.
2	Disambiguation replies ("the second George")	TOOL BUILT	New `resolve_disambiguation_reply` tool. ~250 lines of regex retired. 1 pre-existing inconsistency fixed.
3	Active-subject / pronoun scope ("how many of *them*")	NO TOOL	Evaluated and rejected — state, not text parsing. Hardened with 45 new tests; zero bugs found live.
4	Pending clarifying-question resolution	NO TOOL	No deterministic algorithm exists here at all — nothing to convert. One coverage gap closed.
5	Pagination fast-path + LIMIT-preservation hint	NO TOOL — BUGS FIXED	Converting would make the fail-safe direction *worse*. Found a missing `offset` param that broke all "show more" paging on one path.
6	SQL business-rule hint/enforcement engine (35 rules)	NO TOOL — BUGS FIXED	Table-guard enforcement was silently dead for the entire audit domain. Two independent root causes found and fixed.

3 How Much Regex Was Actually Reduced

This section is deliberately literal, because it's the part most tempting to oversell.

3.1 Eliminated — replaced by explicit tool arguments

Mechanism	Regex/parsing removed
Ordinal reference **PHASE 1**	The mis-scan bug class itself: regex trying to recover *which* noun an ordinal like "the first one" referred to from raw text, including a noun-capturing group prone to grabbing the wrong entity. Replaced entirely — the LLM now supplies `position: int` and an optional `entity_type_hint` directly as tool arguments. The only regex left is a narrow, deliberately dumb backstop (`_ORDINAL_BACKSTOP_RE`, no capturing group) whose sole job is detecting "an ordinal word was used and not resolved yet" — it never tries to parse out *what* anymore.
Disambiguation reply **PHASE 2**	~250 lines removed from `session.py`: filler-word stripping ("the", "um", "I mean"), query-verb gating heuristics, and a multi-tier substring-matching cascade for picking a name out of a reply like "the second one" or "George Smith". Replaced by the LLM supplying `picked_name` / `picked_position` directly.

3.2 Kept, evaluated as correctly regex-based — with real bugs fixed

Mechanism	Why regex stays; what was fixed instead
Pagination fast-path **PHASE 5**	This is the one case where converting to a tool would make things *less* safe, not more — it fully bypasses the LLM on a match, so a false positive here means silently answering the wrong question. The existing regex fails closed by design (whole-message anchored, so ambiguous phrasings safely fall through to the full LLM path). Left as regex; fixed a real bug in the pagination *data path* instead (missing offset — see 4.5).
LIMIT-preservation hint **PHASE 5**	The number being preserved is always a bare digit already sitting in the question text — there is nothing for an LLM to extract that the regex can't already see. Fixed a real false-positive instead: the old pattern matched "last 30 *days*" identically to "last 30 *inspections*".
SQL business-rule INJECT hints **PHASE 6**	These 35 rules match business-vocabulary against the question text the code already has in full — asking an LLM to self-report which of 32 (inspection) or 8 (audit) rules apply to its *own* question would be strictly worse than deterministic matching against text already in hand. Left as regex; fixed two independent enforcement bugs and tightened one over-broad pattern (see Section 4.6).

3.3 Kept, no regex involved to begin with

Mechanism	Why
Active-subject / pronoun scope **PHASE 3**	Driven entirely by `session.current_scope`, a persistent state pointer — no text pattern-matching involved at all.

Pending clarifying-question **PHASE 4**	A trivial `"?" in answer` state check, not a regex. There is no algorithm here to either keep or replace.

> **Net effect:** two genuinely fragile, high-blast-radius regex parsers (ordinal mis-scan, disambiguation reply parsing) are gone, replaced with data the LLM supplies directly and a codebase-wide shared tool framework to support more of these in future. The remaining regex in the system was, on direct inspection, either already the correct tool for its job or actively safer than the tool-call alternative — but three of those "correctly regex" mechanisms turned out to have real, previously-undetected bugs in how they were *enforced*, independent of the parsing question. Those are fixed now.

4 Mechanism by Mechanism — Before vs Now

4.1 Ordinal reference resolution PHASE 0 + 1

"Show me the first one" / "open the second facility"

BEFORE

A regex scanned the raw question for an ordinal word, then tried to infer which entity/noun it referred to from surrounding text — including a noun-capturing group. On the exact historical incident, two facilities shared a display title; the regex's title-based guess silently resolved to the wrong row. The LLM never got a chance to disambiguate — the regex decided unilaterally, upstream of reasoning.

NOW

`resolve_ordinal_reference` is a declared tool. The LLM calls it with a clean `position: int` and optional `entity_type_hint` — data only the LLM can reliably extract from conversational context. A narrow deterministic backstop in `loop.py` blocks `generate_sql` and redirects the model back to the tool if it tries to answer an unresolved ordinal without calling it first.

> **FOUND ONLY BY LIVE TESTING, NOT UNIT TESTS** (1) An import cycle between `tool_defs` and `orchestrator/__init__.py`, fixed with a lazy PEP 562 module `__getattr__`. (2) The real model sends `entity_type_hint` as the schema's actual snake_case table name (`"audit_plan"`), not natural language (`"plan"`) — fixed with underscore-normalization plus a substring fallback, locked in with a regression test.

Live verification: the backstop fired on the model's real first instinct (a title-`ILIKE` guess — the exact original bug shape), correctly redirected, and correctly generated `WHERE ap.id = <uuid>`.

4.2 Disambiguation reply resolution PHASE 2

"the second one" / "George Smith" in reply to a list of same-named candidates

BEFORE

Two separate regex-driven blocks in `session.py` (~250 lines) stripped filler words, gated on query-verb heuristics, and ran a multi-tier substring-matching cascade to guess which candidate a short reply picked. A third, independently-diverging implementation in `synthesis.py` used a different matching rule (exact-whole-question vs. substring) for a related scope-inheritance decision — silently inconsistent with the first two.

NOW

One unified `resolve_disambiguation_reply` tool handles both people-name and entity picks, auto-detecting which kind of disambiguation is pending. The LLM supplies `picked_name` / `picked_position` directly. The genuine data-shape safety case — two real users sharing a display name — is preserved as an explicit STOP, not silently resolved.

BUG FIXED WHILE IMPLEMENTING, NOT A PRE-PLANNED TARGET `synthesis.py`'s independent third implementation was reconciled to prefer the current turn's own resolved tool-call result over re-deriving from text — which also newly covers people-picks correctly for the first time, since only entity picks fed this inheritance path before.

Live verification, two real scenarios: a 45-person real ambiguous search (LLM correctly asked for more detail instead of guessing at "20 of 45 captured"), and the exact confirmed-live case of two real users both named "George Auditor" — the tool returned both UUIDs and correctly refused to pick one.

4.3 Active-subject / pronoun scope PHASE 3

*"how many of **them** are still open" — no entity named in the follow-up*

BEFORE

`session.current_scope` — a persistent pointer set once, then legitimately reused for every subsequent turn of a multi-turn drilldown. Purely session-state driven; no text pattern-matching.

NOW — UNCHANGED, DELIBERATELY

Converting this to a tool would mean calling it on nearly every follow-up turn in a long conversation — a real per-turn cost added to the *common* case, for data the code already has 100% deterministically from state. Kept as-is; hardened instead with 45 new tests (near-zero prior coverage) across all five sub-areas, plus a cosmetic DRY of two duplicate safety-STOP blocks into one shared helper (zero behavior change).

Live verification, three real chained turns: "which facility has the most corrective actions" → "how many of them are still open" (scope inherited, no facility named) → "who is the auditor assigned to them" (scope survived to a *third* consecutive turn). Zero bugs found — this run also validated the "no tool needed" conclusion itself, since the existing mechanism worked correctly live with no changes.

4.4 Pending clarifying-question resolution PHASE 4

The LLM asked its own clarifying question last turn — does this reply answer it?

BEFORE

Fires on a trivial `"?" in previous_answer` check, then unconditionally reminds the LLM of its own earlier question — the actual judgment of "does this reply answer that question" is left entirely to the LLM's own reasoning. No algorithm to speak of.

NOW — UNCHANGED, DELIBERATELY

Connecting a short reply back to an earlier self-invented question isn't reducible to an int or string a tool could return — there was nothing to convert. One real coverage gap closed: confirmed and asserted that `redirect_domain` deliberately does *not* set the clarification-origin field, since a domain handoff breaks the "next message answers this same orchestrator" premise.

Live verification produced a genuinely interesting non-result: two attempts to reproduce the original historical ambiguity both resolved confidently instead of asking a clarifying question — not a bug, but the system's own accumulated ambiguity-resolution rules since the original incident have patched the trigger at a different layer. Existing real-incident-text test coverage was judged sufficient without forcing a synthetic repro.

4.5 Pagination fast-path + LIMIT-preservation hint PHASE 5

"show me more" — continue an existing paged result without re-asking the LLM

BEFORE

A whole-message-anchored regex bypasses the LLM entirely on a match — the one mechanism in this whole effort that skips reasoning altogether rather than just injecting a hint. The `get_answers` pagination code path had **no `offset` parameter anywhere** — not in the function signature, not in its SQL, not on the resulting conversation turn. Every repeated "show more" on that path silently re-fetched the identical first 500 rows, forever. Separately, the LIMIT-preservation regex matched `"last 30 days"` identically to `"last 30 inspections"`, injecting a bogus row-count hint into pure date-filter questions.

NOW

Fast-path regex left as-is (confirmed to fail closed on real phrasings — "please show more", "continue please" all correctly fall through to the safe LLM path rather than misfiring). `get_answers()` gained a real `offset` parameter, threaded end to end, with the same "no more results" short-circuit the other path already had. The LIMIT regex gained a negative lookahead excluding time-unit nouns. A new AST-based `SOFT_WARN` check compares the requested row count against the SQL's actual top-level `LIMIT` (correctly distinguishing an outer LIMIT from one buried in a CTE) — soft, not hard-fail, since legitimate CTE/subquery LIMIT patterns exist.

UNRELATED FINDING, FLAGGED NOT FIXED Every row in `ai_answers` has an `inspection_report_id` matching zero real `inspection_report.id` rows — a dataset-wide orphaned foreign key, discovered while trying to drive a live pagination test. Out of scope for this effort; tracked separately.

4.6 SQL business-rule hint / enforcement engine PHASE 6

The ~35-rule INJECT system — matches question vocabulary to schema-specific SQL guidance and enforcement

BEFORE

Two independent bugs meant the post-generation "table guard" — which is supposed to force a corrective retry when generated SQL violates a fired rule's required tables or forbidden join patterns — **never actually enforced anything for the audit domain, silently, for as long as the code existed.** Root cause #1: the guard hardcoded the inspection-domain rule registry regardless of which domain was running, so every audit-only rule ID looked up empty. Root cause #2, found only while writing the regression test for #1: even after fixing the registry lookup, the guard's retry-prompt construction passed a rule's raw forbidden-pattern regex text (e.g. `\baca\.audit_id\s*=\s*asch\.id\b`) as the *replacement string* to `re.sub()` — Python's replacement-string parser treats `\s` as an invalid escape and raises, silently swallowed by a bare `except Exception`.

NOW

The guard receives the correct per-domain `RulesEngine` instance, threaded from `loop.py` through `ToolRegistry` into `generate_sql()` as a new `_rules_engine` parameter (mirroring the existing `_join_graph_module` pattern). The retry-prompt construction uses a callable `re.sub` replacement instead of a template string, sidestepping the escape parser entirely. A measured budget cap (4,500 characters) was added to the separate hint-injection path so stacked rule prose can't unboundedly grow the prompt — the enforcement half (`required_tables` / `forbidden_patterns`) is deliberately exempt from the cap.

CONFIRMED LIVE, IN REAL PRODUCTION TRAFFIC — NOT JUST SYNTHETIC TESTS The user's own eval-suite run after these fixes shows the table guard firing for the audit domain for the first time ever (`debug_llm_20260805_171956.log`, questions `FORMS-01`/ `FORMS-03`) — structurally impossible before this session's fix. Both fires were themselves a secondary false positive: `audit_inject_audit_type_resolution`'s pattern matched any "audit type" mention, including audit-forms-catalog questions that have no logical join path to the `audit_plan` table its `required_tables` demanded — wasting one LLM call each time, though the model correctly ignored the bogus instruction and both queries still passed. Fixed by narrowing the rule's `question_pattern` with a whole-string negative lookahead excluding form/template context, verified against both the false-positive questions and legitimate report/schedule questions.

Eval-suite regression check (before vs. after, same question sets): inspection domain 95/103 passing both times; audit domain 73/73 passing both times — zero regressions from any of today's changes.

5 Testing & Verification Discipline

Every phase in this session closed the same way: unit tests first, then a live run against the real Postgres database and real Ollama models — never one without the other. Several of the real bugs listed above (the ordinal entity-type-hint format, the audit-domain table-guard being silently dead, the eval-surfaced rule false-positive) were found *only* by the live step; the unit tests alone would have reported green.

Check	Result
Full backend test suite	400 passed, 0 failed
Live DB + LLM verification runs this session	Ordinal (×2), disambiguation (×2), active-scope (3-turn chain), clarification (×2), pagination (SQL-level direct check), table-guard (×2)

Post-session eval suite — inspection domain	95/103 `real_pass`, identical before/after today's changes
Post-session eval suite — audit domain	73/73 `real_pass`, identical before/after today's changes

6 Where This Leaves the Codebase

This was the last item on the original audit's candidate list — every regex-shaped intent-resolution pipeline identified at the start of this effort has now been evaluated. Two became tools; four were kept, correctly, after direct evidence rather than a default assumption that "regex bad, tool good." Three of those four kept mechanisms turned out to have real, independently-discovered defects unrelated to the regex-vs-tool question itself — those are fixed and covered by new regression tests. No further phase is currently queued.